Контест 12 · Нагрузка и масштаб

Неделя 24 · темы недель 22–24: метрики задержки, очереди, ограничение частоты, шардирование и кэш, пул соединений и индексы; в проектной части — расширение по спеке. Сдача устная у экрана: показываете код, запускаете на примерах, объясняете идею.

> Монеты: обычная задача — **2**, со звёздочкой (★, например B4*) — **4**, каждая часть босса — **6**. В этом контесте: 15 обычных, 3 со звёздочкой, 3 частей босса — всего **60** монет. Задача засчитывается, только если вы можете объяснить свой код.

A. Метрики

A1. `rps_series(timestamps)` — сколько запросов пришло в каждую секунду `[s, s + 1)`, от секунды 0 до последней непустой включительно. Пустой список — `[]`.

```
Ввод:  rps_series([0.1, 0.5, 1.2, 3.9, 3.0]), rps_series([])
Вывод: [2, 1, 0, 2], []
```

A2. `apdex(latencies, t)` — индекс Apdex: довольны — задержка `≤ T`, терпят — `≤ 4T`, остальные недовольны. Ответ `(довольны + терпят / 2) / N`, 2 знака. Пустой список — `ValueError`.

```
Ввод:  apdex([100, 200, 300, 500, 900, 1200, 2500], 300), apdex([50] * 9 + [5000], 100)
Вывод: 0.64, 0.9
```

A3. `error_budget(errors, total, slo=0.999)` — бюджет ошибок: разрешено `(1 − slo) · total` ошибок (вниз до целого), `burn = errors / разрешено` (2 знака; при «разрешено 0» — `inf`, если ошибки есть, иначе `0.0`), `left` — сколько ещё можно, не меньше 0. Что значит `burn > 1`?

```
Ввод:  error_budget(30, 100_000), error_budget(5, 1000)
Вывод: {"allowed": 100, "burn": 0.3, "left": 70}, {"allowed": 1, "burn": 5.0, "left": 0}
```

A4*. `hist_percentile(buckets, p)` — гистограмма задержек: пары `(верхняя граница, сколько)` по возрастанию границ. Оценка перцентиля — граница первой корзины, где накопленная доля `≥ p / 100`. Пусто — `ValueError`. Почему так хранят задержки в Prometheus, а не все значения?

```
Ввод:  h = [(10, 50), (25, 30), (50, 15), (100, 4), (250, 1)]
       hist_percentile(h, 50), hist_percentile(h, 95), hist_percentile(h, 99.5)
Вывод: 10, 50, 250
```

B. Очереди

B1. `mm1(lam, mu)` — очередь M/M/1: `λ` запросов в секунду, сервер обрабатывает `μ` в секунду. Загрузка `ρ = λ / μ`, в системе в среднем `L = ρ / (1 − ρ)`, время в системе `W = 1 / (μ − λ)` — в мс. 3 знака. `ρ ≥ 1` — `ValueError`. Во сколько раз растёт `W`, если загрузка с 80 % стала 95 %?

```
Ввод:  mm1(80, 100), mm1(50, 100)
Вывод: {"rho": 0.8, "L": 4.0, "W_ms": 50.0}, {"rho": 0.5, "L": 1.0, "W_ms": 20.0}
```

B2. `queue_waits(arrivals, service, workers)` — очередь FIFO с `workers` обработчиками: запрос `i` приходит в момент `arrivals[i]` (по возрастанию) и обрабатывается `service[i]`. Сколько ждал каждый до начала обработки. Подсказка: куча моментов, когда освободятся обработчики.

```
Ввод:  queue_waits([0, 1, 2, 3, 4], [3] * 5, 1), queue_waits([0, 0, 0, 1], [2] * 4, 2)
Вывод: [0, 2, 4, 6, 8], [0, 0, 2, 1]
```

B3. `token_bucket(rate, capacity, times)` — ограничение частоты «ведро токенов»: в момент 0 ведро полное, копится `rate` токенов в секунду, не больше `capacity`. Запрос в момент `t` (по возрастанию) пропускаем, если есть целый токен, и тратим его. Ответ — список `True` / `False`.

```
Ввод:  token_bucket(1, 3, [0, 0, 0, 0, 0.5, 1, 1, 3, 3, 3, 3])
Вывод: [True, True, True, False, False, True, False, True, True, False, False]
```

C. Распределение и кэш

C1. `moved_mod(keys, n1, n2)` — шардирование `key % n`: какая доля ключей сменит шард при переходе с `n1` на `n2` шардов, 3 знака. Почему с 4 на 8 переезжает меньше, чем с 4 на 5?

```
Ввод:  moved_mod(list(range(10_000)), 4, 5), moved_mod(list(range(10_000)), 4, 8)
Вывод: 0.8, 0.5
```

C2. `Ring(nodes, vnodes=50)` — согласованное хэширование. `_h(s)` — первые 8 hex-цифр `md5(s)` как число. У узла `vnodes` точек `_h(f"{узел}#{i}")`. `ring.node(key)` — узел первой точки `≥ _h(key)` (после последней — снова первая). Сколько из 2000 ключей `room:0…room:1999` переедет при добавлении `api4`, и куда?

```
Ввод:  r = Ring(["api1", "api2", "api3"]); r.node("room:7"), r.node("room:42")
Вывод: "api2", "api3"
```

C3. `lru_hits(capacity, requests)` — кэш LRU: при переполнении выбрасываем ключ, к которому дольше всех не обращались. Ответ — `(попаданий, ключи в кэше от давних к свежим)`.

```
Ввод:  lru_hits(2, list("abacbdab")), lru_hits(3, list("abacbdab"))
Вывод: (1, ["a", "b"]), (3, ["d", "a", "b"])
```

C4*. `SingleFlight(fetch)` — защита от «набега» на базу, когда кэш истёк: одновременные `await sf.get(key)` с одним ключом вызывают `await fetch(key)` один раз и получают один результат (или одно исключение). После завершения следующий `get` снова вызывает `fetch`.

```
Ввод:  await asyncio.gather(*(sf.get("lb") for _ in range(10)), sf.get("x")); calls
Вывод: ["LB"] * 10 + ["X"]; ["lb", "x"]
```

D. База

D1. `n_plus_one(queries, threshold=3)` — поиск N+1 в логе SQL одного запроса к API. Шаблон: строки в кавычках `'…'` и числа → `?`, пробелы схлопнуть. Шаблоны, встретившиеся `≥ threshold` раз: `(шаблон, сколько)`, по убыванию числа, при равенстве — по шаблону.

```
Ввод:  n_plus_one(["SELECT * FROM rooms WHERE owner = 7", "SELECT 1"]
                  + [f"SELECT name FROM users WHERE id = {i}" for i in (3, 5, 9, 12)])
Вывод: [("SELECT name FROM users WHERE id = ?", 4)]
```

D2. `pool_size(rps, queries_per_request, query_ms, headroom=0.3)` — соединений в пуле реплики по закону Литтла с запасом: `⌈rps · запросов · query_ms / 1000 / (1 − headroom)⌉`.

`max_replicas(max_connections, pool, reserved=10)` — сколько реплик выдержит PostgreSQL.

```
Ввод:  pool_size(200, 2, 15), max_replicas(100, 9)
Вывод: 9, 10
```

D3. `pick_index(where_eq, order_by, indexes)` — правило левого префикса: индекс (кортеж колонок) полезен, если его первые колонки (хотя бы одна) — из `where_eq`. Оценка — длина такого префикса, затем «следующая колонка — `order_by`». Лучший — с большей оценкой, при равенстве — короче, затем по имени. Ни один не подходит — `None`.

```
Ввод:  ix = {"ix_player": ("player_id",),
             "ix_player_time": ("player_id", "finished_at"),
             "ix_room_player": ("room", "player_id"), "ix_time": ("finished_at",)}
       pick_index({"player_id"}, "finished_at", ix), pick_index({"player_id"}, None, ix)
       pick_index({"score"}, None, ix)
Вывод: "ix_player_time", "ix_player"
       None
```

E. Проект и алгоритмы

E1. Спека расширения (лекция 24) разбита на задачи по 1–3 часа с критериями готовности; покажите три задачи и тест, написанный до кода.

E2. PR расширения от агента, который вы отклонили или переделали на ревью: что было не так (границы, тест, безопасность) и что изменили.

E3. Расширение на staging за флагом: покажите сценарий из спеки с телефона и выключите флаг — игра работает без расширения.

E4 · босс. Автомасштабирование реплик API. За каждую часть свой плюс.

а) `desired(current, util, target, lo=1, hi=10, tolerance=0.1)` — формула HPA: `ratio = util / target`; если `|ratio − 1| ≤ tolerance` — оставить `current`, иначе `⌈current · ratio⌉`. Ограничить `[lo, hi]`.

```
Ввод:  desired(3, 0.9, 0.6), desired(3, 0.65, 0.6)
       desired(4, 0.1, 0.6), desired(2, 3.0, 0.6)
Вывод: 5, 3
       1, 10
```

б) `stabilize(recs, window)` — в момент `i` реплик столько, сколько максимум рекомендаций за последние `window` моментов: вверх — сразу, вниз — только когда всё окно ниже. Зачем это нужно?

```
Ввод:  stabilize([2, 5, 3, 3, 1, 1, 1], 3)
Вывод: [2, 5, 5, 5, 3, 3, 1]
```

в) `simulate(load, capacity, target=0.6, start=1, window=3, lo=1, hi=10)` — поминутно: реплика обслуживает `capacity` запросов, лишнее теряется; `util = load / (реплики · capacity)`; рекомендация — `desired`, реплики на следующую минуту — максимум последних `window` рекомендаций. Ответ — `{"replicas": по минутам, "dropped": потеряно всего (1 знак), "peak": максимум реплик}`.

```
Ввод:  simulate([50, 300, 300, 300, 100, 50, 50, 50], 100)
Вывод: {"replicas": [1, 1, 5, 5, 5, 5, 5, 2], "dropped": 200.0, "peak": 5}
```

E5*. Предохранитель (circuit breaker) перед внешним сервисом. `run(events, threshold=3, cooldown=10)` — события `(время, успешен ли был бы вызов)`. После `threshold` неудач подряд — «открыт»: `cooldown` секунд отказываем сразу. Потом — один пробный вызов: удача закрывает, неудача снова открывает. Для каждого события: `"-"` — отказано без вызова, `"+"` — успех, `"x"` — неудача.

```
Ввод:  run([(0, False), (1, False), (2, False), (3, True), (5, True),
            (12, False), (13, True), (22, True), (23, True)])
Вывод: "xxx--x-++"
```